

はじめに

開発・動作環境

本SDKは、.NET Standard 2.1をターゲットとしており、以下のフレームワーク/ランタイム環境での利用を想定しています。

- 対応OS：Windows 10 64bit / Windows 11 64bit
- 対応フレームワーク：.NET ≥ 5.0
- 通信要件：
 - 対応インターフェース：Bluetooth 4.1 以上 または USB 2.0 以上
 - デバイス接続：本SDKを利用するためには、対応する脳血流センサデバイスが PC と正常にペアリングされている必要があります。
 - Bluetooth 利用時：Windows 標準の Bluetooth スタックに対応
 - USB 利用時：デバイス用ドライバが正常にインストールされている必要があります。

本SDKのダウンロード

本SDKのダウンロードリンクはこちらになります。**リンク準備中**

基本操作ガイド

信号処理と解析方法

NIRS信号は、脳活動に伴う血液量変化を計測しています。しかし、これには脈波などの脳活動に由来しない信号も含まれます。タスクや刺激提示を開始すると血液量は5～7秒かけて増加し、止めると15～25秒（リカバリ期間）かけてベースラインに戻ります。これがタスク/刺激に関係する脳活動に由来した信号成分です。

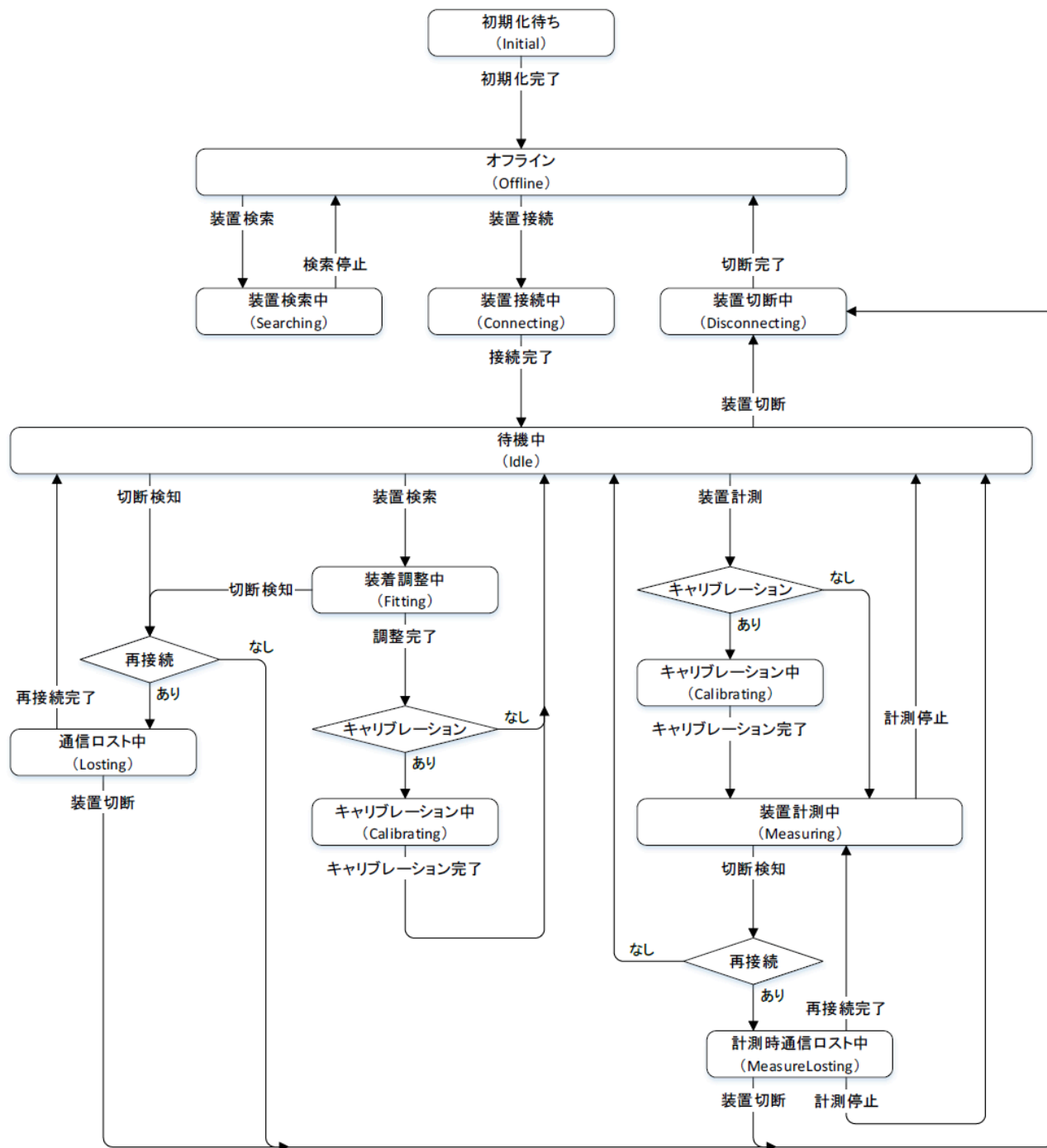
これ以外に、1 Hz程度（安静時）の脈波、0.3Hz程度の呼吸成分がNIRS信号に現れます。脈波と呼吸の成分は、脳活動に由来する信号に比べて高い周波数を有するため、ローパスフィルタを用いて除去できます。更に、メイヤー波（0.1 Hz付近）などの生理活動に伴う低周波の血液量変動成分も存在します。また、脳は常に活動しているので、タスク/刺激に関係しないバックグラウンド脳活動による信号も観測されます。これらの信号を除去するには、ハイパスフィルタや同期検波の手法が用いられます。一般に周波数フィルタを掛けると、信号波形の両端が歪みます。計測時間を十分長くとれない場合には、この歪の影響が無視できなくなるので、ハイパスフィルタではなく、ベースラインを直線近似して減算するなど別の方法が選択されます。

同期検波とは、前レスト期間＋タスク/刺激期間＋後レスト期間を1つのブロックとして、これを繰返し、その繰返しに同期して変動する信号成分を抽出する手法です。ここで、注目するタスク/刺激の無い状態をレストと呼びます。前レスト期間はベースライン補正の方法によって適切な時間を決めます。タスク開始時の1点を用いてベースラインとするならばその1点だけでも構いませんが、数秒程度の前レスト期間を設けてベースライン補正に使います。タスク/刺激時間はその種類によってケースバイケースですが、継続したタスクを行う場合には、上記血液の立ち上がり時間より長くすると信号強度が大きくなります。後レストは上記リカバリ時間にベースライン補正用の時間を加えた期間とします。リカバリ時間が十分に取れない場合は、異なる解析法が必要になりますが本SDKでは対応していません。ここで示した時間は、あくまで目安であり、実際に使うタスク/刺激を用いて計測し、適用可能性と要求精度を考慮して定めるようにして下さい。

得られた信号波形をブロック毎に切り出し加算平均すると、繰返しに同期しない信号成分は位相がずれているため重ね合わせにより減少し、同期する成分、つまりタスク/刺激に関連して変動する信号成分を抽出することができます。更に、レスト期間に比べてタスク/刺激期間の信号が有意に変化しているかどうかを調べるには、平均値差検定を行います。繰返し計測を行わない場合でも、NIRS信号はその計測原理上、ある時点の値からの増量または減少量しか得られませんので、必ず信号の原点を決める必要があります。装置出力は、便宜的に計測開始時を0とすることが多いですが、前述のようにベースラインのゆらぎが大きいため、注目する脳活動を生じさせるタスクまたは刺激の開始前の値または時間平均値を基準に取り直すことが一般的です。High-Level APIは、上記信号前処理と同期検波に対応した信号処理および解析を行なう機能を有します。これまで信号解析は計測後に行うことしかできませんでしたが、本APIを用いることにより、計測中に解析を実施して、その結果を用いて次の計測条件を選択するなど、リアルタイム性を備えたアプリを容易に作成することができます。

動作フロー

ユーザーアプリケーションの観点から本SDKのAPIを使用した主な動作フローを基本例として以下に示します。



NeUライセンスマネージャー

はじめに

本SDKを利用するには発行されたライセンスを受け取り、NeU ライセンスマネージャーによるライセンス有効化が必要になります。

ライセンスが有効化されていない状態で本SDKを利用しようとすると、例外処理が返ってくるような仕様になっております。

NeUライセンスマネージャーの概要

NeU ライセンスマネージャーは、NeU 製品のライセンスを管理するための Windows デスクトップアプリです。複数のライセンスを同時にインストールして使用でき、各ライセンスの状態確認や、有効化・再有効化・無効化を行うことができます。

本ツールでは以下の操作が可能です：

- インストール済みライセンスの一覧表示
- ライセンスの新規有効化
- 既存ライセンスの再有効化（サーバーとの同期）
- ライセンスの無効化
- ライセンス状態の日本語表示
- 手動の再読み込みによる最新状態への更新 ライセンスに関する処理はすべて NeU ライセンスシステムを通して行われます。

主な機能

複数ライセンスの同時管理

複数のライセンスがインストールされている場合、それらを一覧で確認できます。

サーバー連携

有効化・再有効化・無効化は NeU ライセンスサーバーと通信して行われます。

ローカルライセンス情報の確認

ローカルに保存されているライセンス情報（有効期限、GUID、名義など）を表示できます。

ライセンスの内容

一覧に表示される各ライセンスには、以下の情報が含まれます。

ライセンスタイプ ※2026-04-30時点 調整中

- Developer：外部開発者向け
- Commercial：アプリ利用者向け
- Basic：NeUライセンスマネージャーが登場する以前に発行されたTomatoライセンス

ライセンス名義

ユーザーまたは企業名

GUID

ライセンス固有の識別子

有効期限

「無期限」または指定された日付（YYYY/MM/DD）

状態

有効期限切れ/破損/ハードウェア不一致/PC の時刻が正しくありません/その他/不明（英語名が表示されます）

ユーザーインターフェース

画面は以下のように構成されています。

NeUライセンス管理 v1.0

ライセンスタイプ: Basic
ライセンス名義: test1
GUID: 52cd9fd4-0c27-4e4e-a24a-a2653815ffce
有効期限: 2026/12/31
状態: 有効

再有効化
無効化

再読み込み

新しいライセンスを有効化

再読み込み

ライセンス情報を再読み込みし、画面を最新状態に更新します。以下の場合に使用します：

- 外部ツールでライセンスを変更した
- 有効化・無効化後に画面が更新されていない
- 状態を手動で確認したい

新しいライセンスを有効化

ライセンスキー入力ダイアログが開き、入力されたキーをサーバーに送信して以下を行います：

- ライセンスの登録（新規・更新）
- ローカルストアへの保存
- 成功またはエラーの表示

- 一覧の自動更新

再有効化 (各ライセンス上に表示)

サーバーとライセンス情報を再同期します。次の場合に使用します：

- システム時刻を修正した後
- サーバー側の設定が更新された
- ライセンスの状態がおかしいと感じた場合
- 有効期限の延長などが行われた場合

無効化 (各ライセンス上に表示)

ライセンスをローカルから削除し、サーバー上でも無効化します。次のような場合に使用します：

- PC を変更する
- ライセンスを他のユーザーに移したい
- 端末からライセンスを削除したい
- 再度有効化することも可能です（契約内容に依存します）。

リリースライセンス パッケージングガイド

このページは、ExBrain SDK アプリをリリース向けに準備するアプリケーション開発者向けのドキュメントです。

重要なルール

リリースライセンスを作成する際は、ビルドフォルダーではなく、インストール済みのアプリのコピーを NeU License Manager で指定してください。

インストール済みフォルダーがお客様の実行環境です。ビルドフォルダーには余分なファイル、インストーラー出力の欠落、一時バイナリ、デバッグシンボル、または異なるレイアウトが含まれていることがよくあります。誤ったフォルダーを選択した場合、リリース SDK はアプリを **AppInvalid** として拒否する可能性があります。

リリースフロー

1. アプリケーションをビルドします。
2. お客様と同じように、ローカルでインストーラーを実行します。
3. NeU License Manager を開きます。
4. **Create Release License** を選択します。
5. インストール済みアプリケーションフォルダー、またはインストール済みの EXE/スクリプトエントリポイントを選択します。
6. 生成されたリリースパッケージファイルの保存先フォルダーを選択します。
7. NeU License Manager はインストール済みファイルを確認し、ハッシュとアプリ検証 JSON の両方を含む **AppHash** ペイロードを作成します。
8. ツールはその **AppHash** ペイロードを NeU ライセンスサーバーに送信します。
9. サーバーはその **AppHash** ペイロードを含む署名済みリリースライセンスを返します。
10. インストーラーを更新して、生成されたリリースライセンスをレジストリにインストールします。

このプロセスは、出荷するアプリケーションファイルを変更するたびに繰り返してください。

Python アプリ

Python アプリケーションの場合は、出荷するスクリプトとネイティブ依存関係が含まれるインストール済みフォルダーを選択するか、インストール済みエントリスクリプトを選択してください。

実行時は、インストール済みアプリフォルダーを作業ディレクトリとしてアプリを起動するか、そのアプリフォルダーに **ExBrainSdkNative.dll** を配置するか、SDK を読み込む前に **NEU_APP_ROOT** をインストール済みアプリフォルダーに設定してください。これにより、SDK が署名済みリリースライセンスにリストされたファイルをチェックするためのルートフォルダーが提供されます。

リリースアプリのチェックには次のようなファイルが含まれます：

- **.py**、**.pyw**、**.pyz**
- **.pyd**
- **.dll**

- パッケージデータファイル
- アプリと一緒に出荷されるロック/設定/リソースファイル

`__pycache__` や `.pyc` ファイルなどの生成されたキャッシュは除外されます。`log`、`logs`、`temp`、または `tmp` という名前の一般的なトップレベルの可変フォルダーは除外されます。必要な出荷アセットをこれらのフォルダーに入れないでください。

生成ファイル

NeU License Manager は選択した出力フォルダーに次のファイルを書き込みます：

- `NeUReleaseLicense_<license-guid>_instructions_and_license.txt`：バイリンガルのインストーラー手順とリリースライセンス。

この手順ファイルには完全なリリースライセンスが含まれています。インストーラーはそのライセンステキストを示されたとおりに正確に書き込む必要があります。

インストーラーのレジストリキー

エンドユーザー向けリリースビルドの場合は、マシン全体のレジストリ場所にリリースライセンスをインストールしてください：

`HKLM\Software\NeU\Licenses`

次の文字列値を作成します：

- 値の名前：リリースライセンスの GUID
- 値のデータ：手順ファイルの完全なリリースライセンステキスト

`HKLM` への書き込みには管理者権限が必要です。アプリ開発者はアプリケーションインストーラーからこの値を書き込む必要があります。

個別の JSON ファイルや個別の `AppHash` レジストリキーは不要です。アプリ検証 JSON は署名済みリリースライセンスに埋め込まれています。

NeU License Manager を通じてアクティベートされた開発者ライセンスは、引き続き `HKCU\Software\NeU\Licenses` にユーザーごとにインストールされます。SDK は `HKCU` と `HKLM` の両方を検索します。

AppInvalid が発生する原因

リリース SDK は次の場合にライセンスを `AppInvalid` として報告します：

- 埋め込まれたアプリ検証 JSON が存在しない
- 埋め込まれたアプリ検証 JSON の `appHash` が署名済み `AppHash` ペイロードハッシュと一致しない
- 埋め込まれたアプリ検証 JSON が無効なスキーマに編集された
- リストされたファイルが欠落している

- リストされたファイルのサイズが異なる
- リストされたファイルの SHA-256 コンテンツが異なる

SDK は **AppHash** を確認する前に一貫した内部表現を再構築するため、ホワイトスペースと JSON プロパティの順序は問題ありません。

オフライン動作

通常のリリース SDK 機能チェックはインターネットアクセスを必要としません。リリースライセンスは開発者やエンドユーザーのマシン ID に紐付けられていません。インストール済みアプリが署名済み **AppHash** と一致する場合にのみ、任意のマシンで有効です。

エンドユーザーのツールは後から再アクティベーションリクエストを送信することができます。これにより、エンドユーザーの PC をサーバーに表示させることができ、アプリまたはライセンスマネージャーがサーバーと同期を行う際に、リモートで失効したライセンスを検出できます。

リリースライセンスの内部仕様

このページは、ExBrain SDK のリリースライセンス実装を管理しているNeU社向けのドキュメントになります。

概要

リリースライセンスはマシン ID ではなく、特定のアプリバージョンに紐付けられます。署名済みライセンスには **AppHash** フィールドが含まれています。リリースライセンスの場合、NeU License Manager はカノニカルハッシュとアプリ検証 JSON の両方を含む **AppHash** ペイロードを送信します。サーバーはそのペイロードをライセンスに署名します。実行時、SDK は **AppHash** を持つライセンスについて、署名されたペイロードのハッシュが埋め込まれた JSON と一致し、その JSON にリストされたすべてのファイルが記録された SHA-256 ハッシュと一致する場合にのみ有効とみなします。

JSON が存在しない場合、JSON ハッシュが署名済みペイロードハッシュと一致しない場合、またはリストされたファイルのいずれかが欠落または変更されている場合、SDK はライセンスを **AppInvalid** とマークします。

リリースライセンスの検証は意図的にローカルかつオフラインで行われます。実行時の機能チェックはライセンスサーバーを呼び出しません。再アクティベーション/非アクティベーション API は、アプリまたはツールがエンドユーザーの PC をサーバー側に表示させたい場合や、同期中に失効を検出したい場合に引き続きサーバーに接続できます。

アプリバイndingアルゴリズム

検証データは構造化された JSON ですが、ハッシュは生の JSON バイトに対して計算されません。これにより、ホワイトスペースとプロパティ順序の問題を回避しています。

SDK は JSON を解析し、**files** を相対パスでソートした後、次のカノニカルペイロードを再構築します：

```
{ "algorithm": "sha256", "canonicalization": "neu-app-fingerprint-v1", "entryPoint": "...", "files":  
  [ { "path": "...", "sha256": "...", "size": 123 }, { "path": "...", "sha256": "...", "size": 123 }, ... ], "rootType": "directory", "schema": "com.neu.exbrain.app-fingerprint", "schemaVersion": 1 }
```

AppHash は以下のとおりです：

```
sha256(canonical UTF-8 payload)
```

新しいリリースライセンスの場合、サーバーの **appHash** パラメーターに送信される値は JSON 文字列です：

```
{ "schema": "com.neu.exbrain.apphash-v1", "schemaVersion": 1, "hash": "<sha256>", "json": "<app validation JSON>" }
```

サーバーはこの完全な文字列をライセンスの **AppHash** フィールドに署名します。

各ファイルのハッシュは以下のとおりです：

sha256(raw file bytes)

相対パスは常に / を使用し、序数順でソートされます。絶対パスや .. を含むパスは使用できません。

実行時のルックアップ

インストール済みライセンスは両方のレジストリハイブから読み込まれます：

- HKCU\Software\NeU\Licenses
- HKLM\Software\NeU\Licenses

NeU License Manager によってアクティベートされた開発者ライセンスは HKCU に書き込まれます。エンドユーザー向けリリースライセンスは、アプリ開発者のインストーラーによって HKLM にインストールされる必要があります。

SDK はまず、署名済み AppHash ペイロードに埋め込まれた検証 JSON を確認します。古い生成ライセンスとの互換性のため、次の場所で平文の 64 文字 AppHash に対応する検証 JSON も検索します：

- HKCU\Software\NeU\AppFingerprints\<appHash>
- HKLM\Software\NeU\AppFingerprints\<appHash>
- 値 ManifestJson：NeuAppFingerprint.json の完全なコンテンツ
- 値 AppRoot：JSON 相対パスの解決に使用するインストール済みアプリケーションルートディレクトリ
- 環境変数オーバーライド NEU_APP_FINGERPRINT_JSON（オプションで NEU_APP_ROOT）
- プロセス実行ファイルの隣にある NeuAppFingerprint.json
- ExBrainSdkNative.dll の隣にある NeuAppFingerprint.json
- 現在の作業ディレクトリの NeuAppFingerprint.json

埋め込みペイロードライセンスの場合、SDK は実行中の実行ファイルのディレクトリ、ExBrainSdkNative.dll のディレクトリ、現在の作業ディレクトリ、および NEU_APP_ROOT をアプリルートとして試みます。

ライセンス有効性ルール

通常の開発者ライセンスは既存のマシンバインド動作を維持します。

リリースライセンスは AppHash の存在によって検出されます：

- 空の Guid：Corrupted
- 期限切れライセンス：Expired
- AppHash が存在し、JSON/ファイルが一致：Valid
- AppHash が存在するが、JSON/ファイルの検証失敗：AppInvalid
- AppHash なしでマシン ID が不一致：InvalidHardware

リリースライセンスは意図的に MachineId を必要としません。

現在の実装は `MachineId` の前に `AppHash` を確認するため、`MachineId` のないリリースライセンスは検証 JSON とアプリファイルが一致する限り有効です。

High-Level API サンプルアプリ

操作ガイド

1. 装置を起動させて、PC と Bluetooth か USB 接続させて下さい。接続ができていないと計測開始の工程へ進められない仕様になっているため、安定しやすい USB 接続を推奨します。
2. 装置を装着してください。センサーユニットが眉の上に来るように装着し、髪の毛が挟まらないように注意して下さい。具体的な装着方法については、取扱説明書（リンク）をご覧ください。
3. PowerShell を開いていただき、`HighLevelSample.exe` を実行してください。処理がすべて完了すると「Press any key to quit」と表示され、キー入力を受け取るまでアプリは終了しません。これにより、デフォルトのターミナルから実行した場合でもウィンドウが即座に閉じてしまうことを防げます。
4. 実行すると、API 初期化、デバイス接続、ゲイン調整、キャリブレーション、実験開始、複数ブロック計測、統計比較、実験終了、デバイス切断までが順に実行されます。

コード全体

このセクションでは、High-Level API を利用して計測処理を一通り実行するサンプルアプリのコードを掲載しています。コードは、デバイス接続からゲイン調整・キャリブレーション、実験の開始、ブロック実行、統計比較、そして計測終了までの流れを段階的に示しており、実際のアプリケーション開発の基盤となる構造を確認できます。

ここで示されるコードは、後続セクション「各パートのコード解説」で詳しく分解しながら解説します。まずは全体像を把握するために、本セクションを参照してください。

Program.cs

```
using ExBrainSdk;
using ExBrainSdk.Enumerations;

var api = new ExBrainApi((_, s) => Console.WriteLine("C# Status " + s), (id, message,
type) =>
{
    Console.WriteLine($"C# Log: [{id}] [{type}] {message}");
});

_ = api.InitApi(true, false, null, null);
if (await api.SearchAndConnectDeviceAsync(EnmSdkConnectType.eNotSelected, 3)
== EnmSdkResult.eSuccess)
{
    await HighLevelSample.Run(api);
    Console.WriteLine("High-Level Sample finished");
}
else
{
    Console.WriteLine("Failed to connect to device.");
}
```

```

}
api.ExitApi();

Console.WriteLine("Press any key to quit");
Console.ReadKey();

```

HighLevelSample.cs

```

using ExBrainSdk;
using ExBrainSdk.HighLevel;

public static class HighLevelSample
{
    public static async Task Run(ExBrainApi api, Action<string>? log =
null, bool performGainAdjustment = true, bool performCalibration = true,
Experiment.NotifyInstantFrameCallback? instantFrameCallback = null, string? outputFolder
= "")
    {
        log ??= Console.WriteLine;

        if (performGainAdjustment)
        {
            _ = await api.PerformGainAdjustmentAsync(api.HandleId);
            log("Gain adjustment completed.");
        }
        if (performCalibration)
        {
            _ = await api.PerformCalibrationAsync(api.HandleId);
            log("Calibration completed.");
        }

        var experiment = new Experiment(api, 10, instantFrameCallback);
        _ = experiment.SetLowpassFilter(2, 0.2, 10);
        _ = await experiment.StartExperimentAsync(outputFolder);
        log("Experiment started. Idling for 5 seconds...");
        await Task.Delay(TimeSpan.FromSeconds(5));
        log("Idling finished");

        int preFittingSeconds = 5;
        int taskSeconds = 5;
        int calmDownSeconds = 5;
        int postFittingSeconds = 5;
        var preTaskMetaData = new MetaData(api, "TestTask", preFittingSeconds, taskSeconds,
calmDownSeconds, postFittingSeconds);
        var postTaskMetaData = new MetaData(api, "TestTask", preFittingSeconds, taskSeconds,

```

```

    calmDownSeconds, postFittingSeconds);
    int totalSeconds = preFittingSeconds + taskSeconds + calmDownSeconds
+ postFittingSeconds;

    string outputDir = outputFolder ?? "";
    log($"Starting block1 ({totalSeconds} seconds)");
    Experiment.NotifyStageStartedCallback callback = stage =>
    {
        switch (stage)
        {
            case BlockStage.PreFitting:
                break;
            case BlockStage.Task:
                log("TASK");
                break;
            case BlockStage.CalmDown:
                log("REST");
                break;
            case BlockStage.PostFitting:
                break;
            default:
                break;
        }
    };
    log("REST");
    Block block1 = await experiment.RunBlockAsync(preTaskMetaData, "TestBlock",
"Participant1", callback);
    log("Block1 finished");
    await File.WriteAllTextAsync(Path.Combine(outputDir,
"block1.json"), block1.ToJson());
    Block block1_reconstructed = Block.FromJson(api, block1.ToJson());
    await File.WriteAllTextAsync(Path.Combine(outputDir,
"block1.csv"), block1_reconstructed.ToCsv());

    log($"Starting block2 ({totalSeconds} seconds)");
    Block block2 = await experiment.RunBlockAsync(preTaskMetaData, "TestBlock",
"Participant1", callback);
    log("Block2 finished");
    await File.WriteAllTextAsync(Path.Combine(outputDir,
"block2.json"), block2.ToJson());

    log($"Starting block3 ({totalSeconds} seconds)");
    Block block3 = await experiment.RunBlockAsync(preTaskMetaData, "TestBlock",
"Participant1", callback);
    log("Block3 finished");
    await File.WriteAllTextAsync(Path.Combine(outputDir,

```



```

"block3.json"), block3.ToJson());

    // Add your own intervention codes.

    log($"Starting block4 ({totalSeconds} seconds)");
    Block block4 = await experiment.RunBlockAsync(postTaskMetaData, "TestBlock",
"Participant1", callback);
    log("Block4 finished");
    await File.WriteAllTextAsync(Path.Combine(outputDir,
"block4.json"), block4.ToJson());

    log($"Starting block5 ({totalSeconds} seconds)");
    Block block5 = await experiment.RunBlockAsync(postTaskMetaData, "TestBlock",
"Participant1", callback);
    log("Block5 finished");
    await File.WriteAllTextAsync(Path.Combine(outputDir,
"block5.json"), block5.ToJson());

    log($"Starting block6 ({totalSeconds} seconds)");
    Block block6 = await experiment.RunBlockAsync(postTaskMetaData, "TestBlock",
"Participant1", callback);
    log("Block6 finished");
    await File.WriteAllTextAsync(Path.Combine(outputDir,
"block6.json"), block6.ToJson());

    _ = await experiment.FinishExperimentAsync();
    await api.DisconnectDeviceAsync();

    var comparison = TaskComparison.CompareWithinParticipantPrePost(api, [block1,
block2, block3], [block4, block5, block6]);
    log($"The statistical test being used is: {comparison.HbDensityLTestKind}");
    log("Comparison for hbDensityL");
    log($" P-value = {comparison.HbDensityLPValue}, Significant =
{comparison.HbDensityLPValue < 0.05}");
    }
}

```

各パートのコード解説

続いて、各パート毎のコード解説を行っていきます。このサンプルでは、トップレベルの `Program.cs` で初期化と接続を行い、その後 `HighLevelSample.Run(...)` で計測処理全体を実行します。

1. Program.cs の内容

```

using ExBrainSdk;
using ExBrainSdk.Enumerations;

var api = new ExBrainApi((_ , s) => Console.WriteLine("C# Status " + s), (id, message,
type) =>
{
    Console.WriteLine($"C# Log: [{id}] [{type}] {message}");
});

_ = api.InitApi(true, false, null, null);
if (await api.SearchAndConnectDeviceAsync(EnmSdkConnectType.eNotSelected, 3)
== EnmSdkResult.eSuccess)
{
    await HighLevelSample.Run(api);
    Console.WriteLine("High-Level Sample finished");
}
else
{
    Console.WriteLine("Failed to connect to device.");
}
api.ExitApi();

Console.WriteLine("Press any key to quit");
Console.ReadKey();

```

1.1 ExBrainApi の生成

最初に [ExBrainApi のコンストラクタ](#) を呼び出し、API インスタンスを生成しています。コンストラクタにはステータス変化通知用コールバックとログ出力用コールバックを渡しており、SDK 内部の状態変化やメッセージをコンソールへ表示できるようにしています。

```

var api = new ExBrainApi((_ , s) => Console.WriteLine("C# Status " + s), (id, message,
type) =>
{
    Console.WriteLine($"C# Log: [{id}] [{type}] {message}");
});

```

これにより、接続や計測の進行状態をサンプル実行中に確認できます。

1.2 API の初期化

次に [InitApi メソッド](#) を呼び出し、SDK を初期化します。このサンプルでは戻り値を利用しないため、discard である `_` に代入しています。

```
_ = api.InitApi(true, false, null, null);
```

引数では、トレースログ出力を有効にし、通信ログ出力や追加のディレクトリ設定は省略しています。実運用アプリケーションでは、戻り値を確認して失敗時のエラー処理を実装することを推奨します。

1.3 デバイスの検索と接続

初期化後は [SearchAndConnectDeviceAsync メソッド](#) を呼び出して、デバイスの検索と接続を行います。

```
if (await api.SearchAndConnectDeviceAsync(EnmSdkConnectType.eNotSelected, 3)
    == EnmSdkResult.eSuccess)
{
    // 計測処理 (1.4 で解説)
}
else
{
    Console.WriteLine("Failed to connect to device.");
}
```

ここでは `EnmSdkConnectType.eNotSelected` を指定しているため、接続方式を固定せず、利用可能な接続手段から接続を試みる構成になっています。第 2 引数 `3` は検索・接続処理に関するタイムアウト秒数です。接続に成功した場合のみ計測処理 (1.4) を実行し、失敗した場合はメッセージを表示します。いずれの場合も後続の終了処理 (1.5) に進み、キー入力を待ってからアプリが終了するため、接続に失敗してもエラーメッセージを確認できます。

1.4 実験本体の実行

デバイス接続が成功した場合 (1.3 の `if` ブロック内)、実験本体である `HighLevelSample.Run(api)` を呼び出します。

```
await HighLevelSample.Run(api);
Console.WriteLine("High-Level Sample finished");
```

ここでは `HighLevelSample.Run(api)` を呼び出すことで、計測に関する一連の処理を実行し、完了後に終了メッセージを表示しています。

1.5 API の終了処理

最後に [ExitApi メソッド](#) を呼び出して API をクリーンに終了します。

```
api.ExitApi();
```

```
Console.WriteLine("Press any key to quit");
```

```
Console.ReadKey();
```

`api.ExitApi()` は接続の成否にかかわらず（1.3 の `if／else` のどちらを通った場合でも）呼び出され、初期化時に確保された内部リソースを解放します。その後 `Console.ReadKey()` でキー入力を待ってからアプリが終了します。これを最後に置くことで、ターミナルから実行した際にウィンドウが即座に閉じてしまうのを防ぎ、接続失敗時のエラーメッセージも確認できるようにしています。

2. HighLevelSample.Run(...) の内容

```
public static async Task Run(ExBrainApi api, Action<string>? log =
null, bool performGainAdjustment = true, bool performCalibration = true,
Experiment.NotifyInstantFrameCallback? instantFrameCallback = null, string? outputFolder
= "")
{
    log ??= Console.WriteLine;

    if (performGainAdjustment)
    {
        _ = await api.PerformGainAdjustmentAsync(api.HandleId);
        log("Gain adjustment completed.");
    }
    if (performCalibration)
    {
        _ = await api.PerformCalibrationAsync(api.HandleId);
        log("Calibration completed.");
    }

    var experiment = new Experiment(api, 10, instantFrameCallback);
    _ = experiment.SetLowpassFilter(2, 0.2, 10);
    _ = await experiment.StartExperimentAsync(outputFolder);
    log("Experiment started. Idling for 5 seconds...");
    await Task.Delay(TimeSpan.FromSeconds(5));
    log("Idling finished");

    int preFittingSeconds = 5;
    int taskSeconds = 5;
    int calmDownSeconds = 5;
    int postFittingSeconds = 5;
    var preTaskMetaData = new MetaData(api, "TestTask", preFittingSeconds, taskSeconds,
calmDownSeconds, postFittingSeconds);
```

```

    var postTaskMetaData = new MetaData(api, "TestTask", preFittingSeconds, taskSeconds,
    calmDownSeconds, postFittingSeconds);
    int totalSeconds = preFittingSeconds + taskSeconds + calmDownSeconds
    + postFittingSeconds;

    string outputDir = outputFolder ?? "";
    log($"Starting block1 ({totalSeconds} seconds)");
    Experiment.NotifyStageStartedCallback callback = stage =>
    {
        switch (stage)
        {
            case BlockStage.PreFitting:
                break;
            case BlockStage.Task:
                log("TASK");
                break;
            case BlockStage.CalmDown:
                log("REST");
                break;
            case BlockStage.PostFitting:
                break;
            default:
                break;
        }
    };
    log("REST");
    Block block1 = await experiment.RunBlockAsync(preTaskMetaData, "TestBlock",
    "Participant1", callback);
    log("Block1 finished");
    await File.WriteAllTextAsync(Path.Combine(outputDir, "block1.json"), block1.ToJson());
    Block block1_reconstructed = Block.FromJson(api, block1.ToJson());
    await File.WriteAllTextAsync(Path.Combine(outputDir, "block1.csv"),
    block1_reconstructed.ToCsv());

    log($"Starting block2 ({totalSeconds} seconds)");
    Block block2 = await experiment.RunBlockAsync(preTaskMetaData, "TestBlock",
    "Participant1", callback);
    log("Block2 finished");
    await File.WriteAllTextAsync(Path.Combine(outputDir, "block2.json"), block2.ToJson());

    log($"Starting block3 ({totalSeconds} seconds)");
    Block block3 = await experiment.RunBlockAsync(preTaskMetaData, "TestBlock",
    "Participant1", callback);
    log("Block3 finished");
    await File.WriteAllTextAsync(Path.Combine(outputDir, "block3.json"), block3.ToJson());

```

```

// Add your own intervention codes.

log($"Starting block4 ({totalSeconds} seconds)");
Block block4 = await experiment.RunBlockAsync(postTaskMetaData, "TestBlock",
"Participant1", callback);
log("Block4 finished");
await File.WriteAllTextAsync(Path.Combine(outputDir, "block4.json"), block4.ToJson());

log($"Starting block5 ({totalSeconds} seconds)");
Block block5 = await experiment.RunBlockAsync(postTaskMetaData, "TestBlock",
"Participant1", callback);
log("Block5 finished");
await File.WriteAllTextAsync(Path.Combine(outputDir, "block5.json"), block5.ToJson());

log($"Starting block6 ({totalSeconds} seconds)");
Block block6 = await experiment.RunBlockAsync(postTaskMetaData, "TestBlock",
"Participant1", callback);
log("Block6 finished");
await File.WriteAllTextAsync(Path.Combine(outputDir, "block6.json"), block6.ToJson());

_ = await experiment.FinishExperimentAsync();
await api.DisconnectDeviceAsync();

var comparison = TaskComparison.CompareWithinParticipantPrePost(api, [block1, block2,
block3], [block4, block5, block6]);
log($"The statistical test being used is: {comparison.HbDensityLTestKind}");
log("Comparison for hbDensityL");
log($" P-value = {comparison.HbDensityLPValue}, Significant =
{comparison.HbDensityLPValue < 0.05}");
}

```

2.1 ログ出力先の決定

関数冒頭では、呼び出し元からログ出力先が渡されていない場合に `Console.WriteLine` を使用するよう設定しています。

```
log ??= Console.WriteLine;
```

これにより、このサンプルはコンソールアプリとしてそのまま使える一方で、GUI アプリケーションやテストコードから別のログ出力先を差し込むこともできます。

2.2 ゲイン調整の実行

`performGainAdjustment` が `true` の場合、[PerformGainAdjustmentAsync メソッド](#) を呼び出してゲイン調整を行います。

```
if (performGainAdjustment)
{
    _ = await api.PerformGainAdjustmentAsync(api.HandleId);
    log("Gain adjustment completed.");
}
```

`api.HandleId` は接続済みデバイスを識別するためのハンドルです。ゲイン調整は、計測値が適切なレンジに収まるよう補正するための前処理として利用されます。このサンプルでは戻り値を検証していませんが、実際のアプリケーションでは結果コードに基づく分岐処理を追加するのが一般的です。

2.3 キャリブレーションの実行

次に `performCalibration` が `true` の場合、[PerformCalibrationAsync メソッド](#) を呼び出してキャリブレーションを行います。

```
if (performCalibration)
{
    _ = await api.PerformCalibrationAsync(api.HandleId);
    log("Calibration completed.");
}
```

キャリブレーションは装着状態や個人差を反映した補正処理です。ゲイン調整と同様に、このサンプルでは正常系の流れを分かりやすく示すため簡潔な記述にしています。

2.4 Experiment インスタンスの生成とフィルタ設定

計測本体は [Experiment クラス](#) によって管理されます。ここでは `Experiment(api, 10, instantFrameCallback)` を生成し、その後ローパスフィルタを設定しています。

```
var experiment = new Experiment(api, 10, instantFrameCallback);
_ = experiment.SetLowpassFilter(2, 0.2, 10);
```

第 2 引数 `10` は SMD 計算用のターゲットウィンドウサイズです。`instantFrameCallback` を指定すれば、計測中のリアルタイムフレーム通知を受け取ることもできます。また、`SetLowpassFilter(2, 0.2, 10)` では Butterworth ローパスフィルタを設定しています。パラメータは順に、フィルタ次数、カットオフ周波数、サンプリングレートを表します。

2.5 実験の開始

フィルタ設定後、[StartExperimentAsync メソッド](#) を呼び出して計測を開始します。

```
_ = await experiment.StartExperimentAsync(outputFolder);
log("Experiment started. Idling for 5 seconds...");
await Task.Delay(TimeSpan.FromSeconds(5));
log("Idling finished");
```

`outputFolder` を指定すると、実験開始時に CSV 出力先として利用されます。開始直後には 5 秒の待機時間を設けており、ここでは計測開始後の安定化期間や、ユーザーによる導入処理の余地を表現しています。

2.6 ブロック時間設定と MetaData の作成

次に、各ブロックで使用する時間設定を定義し、[MetaData クラス](#) を生成します。

```
int preFittingSeconds = 5;
int taskSeconds = 5;
int calmDownSeconds = 5;
int postFittingSeconds = 5;
var preTaskMetaData = new MetaData(api, "TestTask", preFittingSeconds, taskSeconds,
    calmDownSeconds, postFittingSeconds);
var postTaskMetaData = new MetaData(api, "TestTask", preFittingSeconds, taskSeconds,
    calmDownSeconds, postFittingSeconds);
int totalSeconds = preFittingSeconds + taskSeconds + calmDownSeconds + postFittingSeconds;
```

各値は以下の意味を持ちます。

- `preFittingSeconds`: タスク前の事前装着調整フェーズ
- `taskSeconds`: 実タスクフェーズ
- `calmDownSeconds`: タスク後のクールダウンフェーズ
- `postFittingSeconds`: 最後の事後装着調整フェーズ

このサンプルでは前半と後半で同一設定を使用していますが、`preTaskMetaData` と `postTaskMetaData` を分けることで、将来的に条件の異なるブロック構成へ拡張しやすくしています。

2.7 前半ブロックの実行と出力

最初の 3 ブロックは前半条件として実行されます。各ブロックでは [RunBlockAsync メソッド](#) を呼び出し、完了後に JSON ファイルを保存します。1 ブロック目のみ CSV も併せて出力しています。

```
string outputDir = outputFolder ?? "";
log($"Starting block1 ({totalSeconds} seconds)");
Experiment.NotifyStageStartedCallback callback = stage =>
```



```

{
    switch (stage)
    {
        case BlockStage.PreFitting:
            break;
        case BlockStage.Task:
            log("TASK");
            break;
        case BlockStage.CalmDown:
            log("REST");
            break;
        case BlockStage.PostFitting:
            break;
        default:
            break;
    }
};
log("REST");
Block block1 = await experiment.RunBlockAsync(preTaskMetaData, "TestBlock",
"Participant1", callback);
log("Block1 finished");
await File.WriteAllTextAsync(Path.Combine(outputDir, "block1.json"), block1.ToJson());
Block block1_reconstructed = Block.FromJson(api, block1.ToJson());
await File.WriteAllTextAsync(Path.Combine(outputDir, "block1.csv"),
block1_reconstructed.ToCsv());

log($"Starting block2 ({totalSeconds} seconds)");
Block block2 = await experiment.RunBlockAsync(preTaskMetaData, "TestBlock",
"Participant1", callback);
log("Block2 finished");
await File.WriteAllTextAsync(Path.Combine(outputDir, "block2.json"), block2.ToJson());

log($"Starting block3 ({totalSeconds} seconds)");
Block block3 = await experiment.RunBlockAsync(preTaskMetaData, "TestBlock",
"Participant1", callback);
log("Block3 finished");
await File.WriteAllTextAsync(Path.Combine(outputDir, "block3.json"), block3.ToJson());

```

`RunBlockAsync(...)` は、事前装着調整、タスク、クールダウン、事後装着調整を含む 1 ブロック分の処理を完了まで実行し、その結果を [Block クラス](#) として返します。 `ToJson()` と `ToCsv()` を使うことで、ブロック単位の結果をそのまま保存できます。

`block1` では CSV を出力する前に `Block.FromJson(api, block1.ToJson())` で JSON 経由のラウンドトリップを行い、シリアライズ／デシリアライズが正しく機能することを示しています。通常のアプリケーションでは `block1.ToCsv()` を直接呼び出すだけで十分です。

第 4 引数の [NotifyStageStartedCallback](#) は [BlockStage](#) の各フェーズが開始するたびに [PreFitting](#) / [Task](#) / [CalmDown](#) / [PostFitting](#) の順で通知されます。このサンプルでは `callback` という変数に一度だけ定義し、全 6 ブロックで使い回しています。`switch` 文で受け取ったステージごとに分岐し、[Task](#) 開始時（タスク開始時）に [TASK](#)、[CalmDown](#) 開始時（タスク終了時）に [REST](#) をログ出力することで、参加者に対するタスク／休憩の合図を表現しています。block1 の直前に `log("REST")` を出力しているのは、最初のブロック開始までの初期待機状態を [REST](#) として扱うためです。コールバック自体は省略可能ですが、ここでは全ブロックで同一の挙動が必要なため明示的に渡しています。

2.8 介入処理の挿入ポイント

前半と後半のブロック群の間には、コメントで介入処理の挿入ポイントが示されています。

```
// Add your own intervention codes.
```

ここでは、前半ブロック群と後半ブロック群の間に、刺激提示、条件変更、外部タスク開始などの独自処理を差し込むことを想定しています。

2.9 後半ブロックの実行

後半の 3 ブロックも同様に `RunBlockAsync(...)` で実行し、結果を JSON として保存します。

```
log($"Starting block4 ({totalSeconds} seconds)");
Block block4 = await experiment.RunBlockAsync(postTaskMetaData, "TestBlock",
"Participant1", callback);
log("Block4 finished");
await File.WriteAllTextAsync(Path.Combine(outputDir, "block4.json"), block4.ToJson());

log($"Starting block5 ({totalSeconds} seconds)");
Block block5 = await experiment.RunBlockAsync(postTaskMetaData, "TestBlock",
"Participant1", callback);
log("Block5 finished");
await File.WriteAllTextAsync(Path.Combine(outputDir, "block5.json"), block5.ToJson());

log($"Starting block6 ({totalSeconds} seconds)");
Block block6 = await experiment.RunBlockAsync(postTaskMetaData, "TestBlock",
"Participant1", callback);
log("Block6 finished");
await File.WriteAllTextAsync(Path.Combine(outputDir, "block6.json"), block6.ToJson());
```

後半ブロックも前半と同じ `callback` を使い回すことで、[TASK](#) / [REST](#) の合図表示を全ブロックで統一しています。前半ブロック群と後半ブロック群を分けて保存することで、解析時に条件ごとのまとまりを維持しやすくなります。

2.10 実験の終了とデバイス切断

すべてのブロック実行後、[FinishExperimentAsync メソッド](#) で実験を終了し、その後 [DisconnectDeviceAsync メソッド](#) でデバイスを切断します。

```
_ = await experiment.FinishExperimentAsync();
await api.DisconnectDeviceAsync();
```

計測を終了する際には `FinishExperimentAsync()` を呼び出し、その後にデバイス切断処理を行います。

2.11 介入前後の統計比較

最後に、前半 3 ブロックと後半 3 ブロックを [TaskComparison.CompareWithinParticipantPrePost メソッド](#) で比較します。

```
var comparison = TaskComparison.CompareWithinParticipantPrePost(api, [block1, block2,
block3], [block4, block5, block6]);
log($"The statistical test being used is: {comparison.HbDensityLTestKind}");
log("Comparison for hbDensityL");
log($" P-value = {comparison.HbDensityLPValue}, Significant = {comparison.HbDensityLPValue
< 0.05}");
```

この比較は、同一参加者の介入前後ブロック群を対象にした統計評価です。ログには使用された検定種別と `hbDensityL` の p 値、有意差判定結果を出力しています。必要に応じて右側データや差分値の出力を追加することも可能です。

以上で、High-Level API サンプルアプリのソースコード解説を終わります。

Low-Level API サンプルアプリ

操作ガイド

1. 装置を起動させて、PCとBluetooth かUSB 接続させて下さい。
2. 装置を装着してください。センサーユニットが眉の上に来るように装着し、髪の毛が挟まらないように注意して下さい。具体的な装着方法については、取扱説明書([リンク](#))をご覧ください。
3. PowerShell を開いていただき、`LowLevelSample.exe`を実行してください。処理がすべて完了すると「Press any key to quit」と表示され、キー入力を受け取るまでアプリは終了しません。これにより、デフォルトのターミナルから実行した場合でもウィンドウが即座に閉じてしまうことを防げます。
4. 実行開始すると、API 初期化、装置検出・選択・接続、（任意）ゲイン調整、（任意）キャリブレーション、計測開始、計測停止、装置切断の順に処理が進行します。

また、装置の選択時とゲイン・キャリブレーション開始時、計測開始時、計測終了時の進行をコマンドインターフェースで進める仕様になっております。

コード全体

このセクションでは、Low-Level API を利用して計測処理を一通り実行するサンプルアプリのコードを掲載しています。

```
using ExBrainSdk;
using ExBrainSdk.Datas;
using ExBrainSdk.Enumerations;

internal class Program
{
    private static void Main()
    {
        var api = new ExBrainApi( (_, s) => Console.WriteLine("C# Status " + s), (id,
message, type) =>
        {
            Console.WriteLine($"C# Log: [{id}] [{type}] {message}");
        });

        Sample(api).Wait();
        api.ExitApi();

        Console.WriteLine("Low-Level Sample finished");
        Console.WriteLine("Press any key to quit");
        _ = Console.ReadKey();
    }

    public static async Task Sample(ExBrainApi api)
    {
```

```

EnmSdkResult sdkResult = api.InitApi(
    needOutputTraceLog: true,
    needOutputCommLog: false, // Not implemented
    logDir: null,
    settingsDir: null, // Not implemented
    tag: "" // Not implemented
);
if (sdkResult != EnmSdkResult.eSuccess)
{
    Console.WriteLine("Failed to initialize API.");
    return;
}

await api.ResetBluetoothModuleAsync();

var searchParam = new ExBrainSearchParameter()
{
    Keywords = ["HOT", "XB"],
    ShouldUSB = true,
    ShouldPairedBluetooth = true,
    ShouldNonPairedBluetooth = true,
    TimeoutSecond = 3
};
api.StartSearchDevice(searchParam);
await Task.Delay(3000);
api.StopSearchDevice();
if (api.DiscoveredDevice.DeviceCount == 0)
{
    Console.WriteLine("No devices found.");
    return;
}
else
{
    IReadOnlyList<DeviceInformationEx> devicesInfo = api.DiscoveredDevice.Devices;
    for (int i = 0; i < devicesInfo.Count; i++)
    {
        Console.WriteLine($"Find Device #{i}: Type: {devicesInfo[i].ConnectType},
DeviceId: {devicesInfo[i].DeviceId}");
    }
    Console.WriteLine("Please select a device index to connect:");
    string? index = await Console.In.ReadLineAsync();
    if (!int.TryParse(index, out int selectedIndex) || selectedIndex < 0 ||
selectedIndex >= devicesInfo.Count)
    {
        Console.WriteLine("Invalid index.");
        return;
    }
}

```

```

    }
    else
    {
        DeviceInformationEx selectedDeviceInfo = devicesInfo[selectedIndex];
        sdkResult = await api.ConnectDeviceAsync(selectedDeviceInfo.DeviceId,
selectedDeviceInfo.ConnectType);
        if (sdkResult != EnmSdkResult.eSuccess)
        {
            Console.WriteLine("Failed to connect to device.");
            return;
        }
    }
}

do
{
    Console.WriteLine("Perform Gain Adjustment? (y/n):");
    string? input = await Console.In.ReadLineAsync();
    if (input?.ToLower() == "y")
    {
        sdkResult = await api.PerformGainAdjustmentAsync(api.HandleId);
        if (sdkResult != EnmSdkResult.eSuccess)
        {
            Console.WriteLine("Gain adjustment failed.");
            await api.DisconnectDeviceAsync();
            return;
        }
        Console.WriteLine("Gain adjustment completed.");
        break;
    }
    else if (input?.ToLower() == "n")
    {
        break;
    }
}
while (true);

do
{
    Console.WriteLine("Perform Calibration? (y/n):");
    string? input = await Console.In.ReadLineAsync();
    if (input?.ToLower() == "y")
    {
        sdkResult = await api.PerformCalibrationAsync(api.HandleId);
        if (sdkResult != EnmSdkResult.eSuccess)
        {

```

```

        Console.WriteLine("Calibration failed.");
        await api.DisconnectDeviceAsync();
        return;
    }
    Console.WriteLine("Calibration completed.");
    break;
}
else if (input?.ToLower() == "n")
{
    break;
}
}
while (true);

EnmSdkMeasurementCycle measurementCycle;
do
{
    Console.WriteLine("Select measurement cycle:");
    Console.WriteLine(" 0: 10Hz");
    Console.WriteLine(" 1: 50Hz");
    Console.WriteLine(" 2: 100Hz");
    Console.WriteLine(" 3: 500Hz");
    string? input = await Console.In.ReadLineAsync();
    if (input == "0")
    {
        measurementCycle = EnmSdkMeasurementCycle.e10Hz;
        break;
    }
    if (input == "1")
    {
        measurementCycle = EnmSdkMeasurementCycle.e50Hz;
        break;
    }
    if (input == "2")
    {
        measurementCycle = EnmSdkMeasurementCycle.e100Hz;
        break;
    }
    if (input == "3")
    {
        measurementCycle = EnmSdkMeasurementCycle.e500Hz;
        break;
    }

    Console.WriteLine("Invalid selection. Please enter 0, 1, 2, or 3.");
}

```

```

while (true);

EnmSdkMeasurementMode measurementMode = EnmSdkMeasurementMode.eNormal;
if (measurementCycle != EnmSdkMeasurementCycle.e10Hz)
{
    do
    {
        Console.WriteLine("Select measurement mode:");
        Console.WriteLine(" 0: Low Precision / High Reliability");
        Console.WriteLine(" 1: High Precision / Low Reliability");
        string? input = await Console.In.ReadLineAsync();
        if (input == "0")
        {
            measurementMode = EnmSdkMeasurementMode.eLowPrecisionHighReliability;
            break;
        }
        if (input == "1")
        {
            measurementMode = EnmSdkMeasurementMode.eHighPrecisionLowReliability;
            break;
        }

        Console.WriteLine("Invalid selection. Please enter 0 or 1.");
    }
    while (true);
}

List<ExBrainMeasureData> frames = [];
sdkResult = await api.StartMeasureAsync(api.HandleId, "./", (_, data) =>
{
    frames.Add(data);
}, measurementCycle: measurementCycle, measurementMode: measurementMode);
if (sdkResult != EnmSdkResult.eSuccess)
{
    Console.WriteLine("Failed to start measurement.");
    await api.DisconnectDeviceAsync();
    return;
}

do
{
    Console.WriteLine("Stop Measurement? (y/n):");
    string? input = await Console.In.ReadLineAsync();
    if (input?.ToLower() == "y")
    {
        break;
    }
}

```



```
        }  
    }  
    while (true);  
  
    sdkResult = await api.StopMeasureAsync(api.HandleId);  
    if (sdkResult != EnmSdkResult.eSuccess)  
    {  
        Console.WriteLine("Failed to stop measurement.");  
    }  
    await api.DisconnectDeviceAsync();  
}  
}
```

出力CSVデータ

CSVデータの項目について

計測が終了するとCSVデータが出力されます。

Low-Level API による基本データを記録したCSVファイルと、High-Level API による解析結果を含むCSVファイルとの2種類が存在します。

それぞれのCSVファイルにおける、データ項目について説明します。

Low-Level CSV

Low-Level APIから出力されるCSVデータの項目です。

計測モードによるデータの違い

計測サイクル（10Hz / 50Hz / 100Hz / 500Hz）によって、1フレームあたりのデータ内容が異なります。

- 10Hz モード: 全項目が出力されます。
- 50Hz / 100Hz / 500Hz モード（高速サンプリングモード）: 各フレームには左チャンネルの生体光センサーデータのみが含まれます。右チャンネルデータ・IMUデータ・飽和カウンタ・カラーインデックス・心拍数・ノイズ検出フラグは常に0となります。BatteryVolt / BatteryRemain は計測開始時点の値が全フレームに固定で記録されます。また、このモードではCSVファイル名の末尾に `_fast` が付加されます。

下表の「高速サンプリング」列は、50Hz / 100Hz / 500Hz モードでの出力可否を示します（○: 有効値あり、0: 常に0）。

項目名	説明	高速サンプリング
DeviceTime	計測時の年月日時分秒.ミリ秒 [yyyy/MM/dd HH:mm:ss.fff]	有効
HeadsetTime(sec)	開始時刻からの経過時間 [秒]	有効
BrainIndex(left)	脳活動値（左） [-]	有効
BrainIndex(right)	脳活動値（右） [-]	常に0
PulseRate	心拍数 [bpm]	常に0
dTHbL(leftSD1cm)	ヘモグロビン濃度変化（左 SD1cm） [-]	有効
dTHbL(leftSD3cm)	ヘモグロビン濃度変化（左 SD3cm） [-]	有効
dTHbL(rightSD1cm)	ヘモグロビン濃度変化（右 SD1cm） [-]	常に0

項目名	説明	高速サンプリング
dTHbL(rightSD3cm)	ヘモグロビン濃度変化（右 SD3cm） [-]	常に0
Saturation(leftSD1cm)	飽和カウンタ（左 SD1cm） [回]	常に0
Saturation(leftSD3cm)	飽和カウンタ（左 SD3cm） [回]	常に0
Saturation(rightSD1cm)	飽和カウンタ（右 SD1cm） [回]	常に0
Saturation(rightSD3cm)	飽和カウンタ（右 SD3cm） [回]	常に0
ColorIndex(left)	カラーインデックス（左） [-]	常に0
ColorIndex(right)	カラーインデックス（右） [-]	常に0
NoiseDetectionFlag	ノイズ検出フラグ [-]	常に0
Mark	マーカー管理機能により、被験者の状態やタスクシーケンスの内容に関する結果を記録する。項目間を\ で結合させています。	有効
Acc-X	加速度 X [G]	常に0
Acc-Y	加速度 Y [G]	常に0
Acc-Z	加速度 Z [G]	常に0
Gyro-X	角速度 X [dps]	常に0
Gyro-Y	角速度 Y [dps]	常に0
Gyro-Z	角速度 Z [dps]	常に0
BatteryVolt	バッテリー電圧 [V]	有効 (計測開始時の値で固定)
BatteryRemain	バッテリー残量 [%]	有効 (計測開始時の値で固定)

High-Level CSV

High-Level APIから出力されるCSVデータの項目です。Low-Level CSVとは独立したファイルで、解析済みデータを含みます。

項目名	説明
DeviceTime	計測時の年月日時分秒.ミリ秒 [yyyy/MM/dd HH:mm:ss.fff]
HeadsetTime(sec)	開始時刻からの経過時間 [秒]
BlockName	ブロック名
Identity	識別子
StageName	ステージ名 (PreFitting / Task / CalmDown / PostFitting)
BrainIndex(left)	脳活動値 (左) [-]
BrainIndex(right)	脳活動値 (右) [-]
ColorIndex(left)	カラーインデックス (左) [-]
ColorIndex(right)	カラーインデックス (右) [-]
PulseRate	心拍数 [bpm]
dTHbL(leftSD1cm)	ヘモグロビン濃度変化 (左 SD1cm) [-]
dTHbL(leftSD3cm)	ヘモグロビン濃度変化 (左 SD3cm) [-]
dTHbL(rightSD1cm)	ヘモグロビン濃度変化 (右 SD1cm) [-]
dTHbL(rightSD3cm)	ヘモグロビン濃度変化 (右 SD3cm) [-]
SyntheticAcc	合成加速度 [G]
Acc-X	加速度 X [G]
Acc-Y	加速度 Y [G]
Acc-Z	加速度 Z [G]
Gyro-X	角速度 X [dps]
Gyro-Y	角速度 Y [dps]
Gyro-Z	角速度 Z [dps]
ElapsedSec	カウント開始からの経過時間 [秒]

項目名	説明
DropCount	ハードウェア側カウンタについて、現在のと前回の差から1を引いた値（ハードウェア側でドロップされたフレーム数のみを考慮）。[フレーム]
Mark	マーカー管理機能により、被験者の状態やタスクシーケンスの内容に関する結果を記録する。項目間を\ で結合させています。
dTHbL_processed(leftSD3-1cm)	処理済みヘモグロビン濃度変化（左 SD3-1cm） [-]
dTHbL_processed(rightSD3-1cm)	処理済みヘモグロビン濃度変化（右 SD3-1cm） [-]
SmdBrainIndex(left)	標準化平均差・脳活動値（左）(Standardized Mean Difference) [-]
SmdBrainIndex(right)	標準化平均差・脳活動値（右）(Standardized Mean Difference) [-]

トラブルシューティング

Q&A

Bluetoothでそもそも装置が検出されない場合は？

確認事項：

- Bluetooth機能はONになっていますか？
- 装置が他のBluetooth機器とペアリングされていませんか？
- 他のアプリケーションが、既に装置とペアリングを行っていませんか？
- お使いのPCはBluetoothに対応していますか？

対処方法：

- 装置の電源を切り、次にアプリケーションを完全に終了し、「設定」から「Bluetoothの設定」を確認し、装置がペアリング前の状態でリストに出ていることを確認します。
 - 既にペアリング済みとなっている場合は、一度接続を切断してからアプリケーションを立ち上げ直してください。
-

Bluetooth接続が頻繁に切れる場合は？

確認事項：

- 周囲の電波状況は安定していますか？ Wi-Fi電波やBluetooth機器がたくさん電波を発している環境ではペアリングが頻繁に切れる可能性があります。
- 電源が切れていたり、バッテリーの残量が少なくなっていないですか？

対処方法：

- 電波環境の良い場所に移動します。もしくは、計測に不必要なネットワーク機器を使用しないようにします。
 - バッテリー残量が少ない場合は充電してください。
 - 電源が切れているときは、モバイルバッテリーやAC アダプター等を接続し計測することも可能です。
-

ゲイン調整で何度も失敗してしまう場合は？

確認事項：

- ヘッドセットは正しく装着されていますか？ 髪の毛がはさまっていたり、緩く取り付けられていたり、汗を多くかいていたりとゲイン調整に失敗することがあります。

対処方法：

- ヘッドセットを正しく装着します。稀に前額部の形状によっては、センサーユニットが額に密着しない方がいらっしゃいます。センサーユニットの位置を微調整する等して、密着しやすい箇所での計測をお試しください。
-

サポート窓口

上記以外のケースで解決されない場合は、大変お手数ですが下記窓口までお問合せいただければと思います。

- [お問い合わせフォーム（株式会社NeU）](#) 

NIRSハードウェア & システム（研究用機器）を選択してください。

利用規約

※Comming soon...

その他

参考資料

本SDKをご利用になるうえで、関連する参考資料になります。

- 取扱説明書 研究開発向け 携帯型脳活動計測装置(型式：HOT-2000)

サードパーティ利用状況

本SDKは以下のサードパーティライブラリを使用しています。

Ring Buffer

- リポジトリ: <https://github.com/jnk0le/Ring-Buffer>
- 著作権: Copyright (c) 2017-2022 Jan Oleksiewicz
- ライセンス: MIT License

内部のデータ通信に使用するリングバッファの実装です。C++11のみで実装されており、OS依存がありません。ロックフリー・ウェイトフリーなSPSC（Single Producer Single Consumer）動作をサポートし、動的メモリ確保や例外を一切使用しない軽量な実装です。

MITライセンスの全文は以下の通りです。

MIT License

Copyright (c) 2022 (2017-2022 @CC0) Jan Oleksiewicz

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

Simple-FFT

- リポジトリ: <https://github.com/d1vanov/Simple-FFT>
- 著作権: Copyright (c) 2013-2020 Dmitry Ivanov
- ライセンス: MIT License

脳波データの周波数解析に使用する高速フーリエ変換（FFT）の実装です。ヘッダーオンリーのC++ライブラリで、1次元・2次元・3次元データのFFTに対応しています。Radix-2 Cooley-Tukey アルゴリズムを採用しており、外部依存なしで利用できます。なお、変換対象のデータサイズは2の累乗である必要があります。

MITライセンスの全文は以下の通りです。

Copyright (c) 2013-2020 Dmitry Ivanov

The MIT License (MIT)

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.